Hey Ron! Let’s kick off your practice today.

This is your first course, so everything is brand new and exciting.

Are you ready to dive into some Python practice to prepare for your weekly review?

Monday
Sure am

Let's go over the main topics you learned in Introduction to Python programming:

Writing your first program: print(), variables, data types, math with numbers, and modeling real-world calculations.
Core foundations: division, modulo, rounding, formatting, type checks and conversions, user input, handling Python errors, and building reusable functions.
Decisions and loops: if/else, logical operators, booleans, membership tests, for and while loops, loop control, counters, and totals.
Summary and deeper skills: string slicing, string methods, creating and working with lists, list iteration, list methods, and sorting.
Is there a specific topic you want to focus on, or should I give you a random mix from all areas?

Random please

Great, I’ll set it up with a random mix from all areas.

How much time do you want to dedicate to practice? I recommend anywhere from 15 to 60 minutes.

30 to 60is fine

Perfect!

What difficulty would you like—easy, medium, hard, or a mix of all levels?

Hard is okay

Awesome, here’s what I have for your practice session:

Focus: Random mix of all Python topics from the course
Time: 30 to 60 minutes
Difficulty: Hard
Is this all correct? Let me know if you want to add or change anything, or confirm if you’re good to go!

yes

Perfect, your goals are set! I’m preparing your personalized hard-level practice session now—it should be ready in less than a minute.

You’ve got this, Ron!

Here’s how our session will work today:

We’ll tackle a mix of tough Python problems covering strings, lists, loops, logic, and more.
You’re in charge. If you want to slow down, speed up, skip, or go deeper, just say so at any time.
It’s totally normal to make mistakes, especially with numbers, indexing, or slicing—if something feels off, let me know. We’ll tackle it together.
Does all that sound good? Ready for your warm-up?

yes

Let’s start with a warm-up on string slicing and indexing.

What will this print exactly?

Python

```
text = "Unexpected"
print(text[1::2])
```

Say your answer out loud or type it—no running the code yet!

nxetd ots start at 2nd letter (position 1) and grab every 2nd char till the end

Nice explanation! You got the logic: start at the 2nd letter (position 1) and step by 2 until the end.

The exact output is the string formed by those characters, left to right.

Can you type the exact output you expect, letter for letter, no spaces or quotes?

nxctd sorry had typo in first response

Good thinking on the pattern!

Let’s check each character for clarity:

```
text = "U n e x p e c t e d"
Indexes: 0 1 2 3 4 5 6 7 8 9
```

Starting at index 1, picking every second letter:

Index 1 (n), 3 (x), 5 (e), 7 (t), 9 (d)

So, what full string do you get if you join those? Give it one more try!

nxetd sorry stupid keyboard has a few sticking keys

No worries at all—happens to everyone!

And you got it: the output is nxetd.

Ready to jump into the next challenge?

yeah

Alright, let’s mix numbers, input, and string formatting.

Write a one-liner program that:

Asks the user for a price,
Adds 7.5% tax,
Prints this: Total: $xx.xx (with two decimals).
When you’re ready, use Run & Submit to share your code.

Python

```
exit_flag = True
price = 0.0
while (exit_flag == True):   # Get inputf rom the user , dont move on until a valid entry is received
    try:
        price = float(input("Pleaes enter a price: $"))
        exit_flag = False
    except ValueError:
        exit_flag = True
        print("Invalid input. Please enter a numeric value.")
```

```
total_price = (price * .075) + price
print(f"Total: ${total_price:.2f}")
```

Nice job building in user input validation! Your code handles errors, calculates tax, and prints the total in the correct format. That’s a step beyond what was required.

For this challenge, though, you only needed a single line to:

take input,
convert to float,
add 7.5%,
print the total with two decimals,
all in one line.
Let’s simplify:

Can you write a one-liner that does everything in one print() statement? Give it a try!

stumped i show you what i got but its not adding the oriignal entered price to the calculated ta as price = float(input"")) ist allowd in a fstring

Great question—Python syntax won’t let you declare a variable inside an f-string.

But you can nest the float(input()) calculation directly inside the math part of the f-string. Like this, all in one line:

Python

```
print(f"Total: ${(x * 1.075):.2f}")
```

Instead of x, you can put float(input("...")) right inside.

Try writing that one-liner using this idea. Give it a shot!

Python

```
print(f"Total: $" +
    f"{(float(input("Pleae enter a price $ ")) *.075):.2f}")
```

Nice effort making it short! Your current version only calculates 7.5% of the price, not the total (original price + tax).

Remember, you want to multiply the entered value by 1.075 (not just 0.075).

Try adjusting your formula so it adds the tax to the original price.

Ready to fix and resubmit?

thaasts wat i aid

You’re on the right path!

You only need to change *.075 to *1.075 inside your code.

Would you like to update your code and give it one more go?

hmm thtats a math error not coding rusty didn't know 1.075 * value is == value + (value * .075)

Exactly, it’s just a math shortcut!

value * 1.075 means you’re taking the original value (100%) and adding 7.5% tax, so it’s the same as value + (value * 0.075).

Want to run your fixed one-liner and submit?

Python

```
print(f"Total: $" +
    f"{(float(input("Pleae enter a price $ ")) * 1.075):.2f}")
```

That’s it! You nailed it—your one-liner works just as needed.

It asks for a price,
Calculates with tax,
Prints the result as Total: $10.75 for input 10.
Well done. Want to try a slightly trickier math/logic challenge next, or would you rather switch topics?

can try ather

Alright, let’s go for the next one—this one’s about loops, functions, and returning the right result.

Write a function called next_bus(waited_min) that:

Returns "now" if waited_min % 15 == 0
Otherwise returns the number of minutes left until the next 15-minute departure
Test it with:

print(next_bus(7)) ➔ should print 8

print(next_bus(30)) ➔ should print "now"

print(next_bus(41)) ➔ should print 4

When ready, submit your code with Run & Submit!

Python

```
def next_bus(waited_min):
    if (waited_min % 15 == 0):
        return("now")
    else:
        return(str(15 - (waited_min % 15)))
print(next_bus(7))
print(next_bus(30))
print(next_bus(41))
```

Nice work! Your function returns the correct results for all test inputs:

7 → 8
30 → "now"
41 → 4
Technically, you returned "8" and "4" as strings, but since they were printed, the output matches what we wanted. For perfect style, you can return them as integers instead. Want to adjust that, or keep moving forward?

as you said return "now" on firt entry wich i a string why hot retrn trings for the other returns

Great question! Returning "now" as a string makes sense because it’s a word, not a number.

For consistency, though, Python style usually matches return types in a function. So if the function sometimes returns a string and sometimes an integer, it can be confusing to someone reading the code.

Passing the printed value works fine, but in a real project, developers might expect the return type to always be the same (all strings, or all ints, or always a str for the user, etc.).

But for this challenge, either works as long as you’re clear. Want to practice it again with all returns as strings, or all as ints (except “now”)? Or should we move to the next challenge?

id never code tht way functions should be cositant on what they retrn

That’s exactly right—functions should have consistent return types.

Mixing types is usually a sign to rethink the design.

Great software instinct!

Ready to tackle the next list mutation and median challenge?

sure

Here’s the next challenge:

Start with this list:

Python

```
scores = [88, 91, 74, 88, 100]
```

Do these steps:

Remove the lowest score.
Append 93.
Print the median of the sorted list.
When you're ready, use Run & Submit.

Python

```
scores = [88, 91, 74, 88, 100]
low_num = 100
median_val = 0
for i in range(0,len(scores)):
  if scores[i] < low_num:
    low_num = scores[i]
scores.remove(low_num)
scores.append(93)
sorted_list = sorted(scores)
if (len(sorted_list) % 2 == 0):
    median_val = (sorted_list[int(len(sorted_list)/2)] +
    sorted_list[int(len(sorted_list)/2+1)])  / 2
else:
    median_offs = int((len(sorted_list) + 1) / 2)
    median_val = sorted_list[median_offs-1]
print(sorted_list)
print(median_val)
```

Great job! Your code:

Removes the lowest score
Appends 93
Sorts the list
Prints the correct median (91)
A small note: your median logic works for odd-length lists, but for even-length lists, the formula would need a tweak (not a problem here, since the list is odd after removal and append).

Are you comfortable with how you got the median, or want a quick tip to simplify that? Or shall we move to the next challenge?

what tweak

Great follow-up!

For even-length lists, the median is the average of the two middle numbers.

Here’s the classic approach:

For odd length: middle is len // 2
For even length: average the two values at len//2 - 1 and len//2
Example:

Python

```
sorted_ = [3, 5, 7, 9] # len = 4 (even)
mid = len(sorted_) // 2
```

```
median = (sorted_[mid - 1] + sorted_[mid]) / 2  # (5+7)/2 = 6.0
```

Does that make sense?

yeah had to lookup how to do a median been 30 years since a math class i did get that forula but got ot n coding it and forget i eeded the average ofthos 2 middle valuess my bad

No worries at all—the logic you used got you there, and it worked for the odd-length list.

It's normal to need a quick math refresher after a break!

What matters is: you're problem-solving and making progress. That's what counts.

Ready for the next challenge—a mini text analyzer?

sure

Here's your next one:

Write code that asks for a sentence, then prints:

a) The total word count

b) The number of vowels

c) The longest word

Tip: Use .split(), loops, and string comparisons.

When ready, use Run & Submit!

Today
Python

```
longest_word = 0
vowel_count = 0
sentance = input("Please type in a random sentance and hit enter: ")
for i in range(0, len(sentance)):
    if sentance[i] in "aeiou":
       vowel_count += 1
my_list = sentance.split(" ")
word_count = len(my_list)
for i in range(0 , len(my_list)):
    if len(my_list[i]) > longest_word:
        longest_word = len(my_list[i])
print(f"\nTotal Word Count: {word_count}\n"
      f"Number Of Vowels: {vowel_count}\n"
      f"The Longest Word: {longest_word}"
      )
```

Very nice! You:

Counted words
Counted vowels
Checked for the longest word (by length)
There's one tiny improvement:

Currently, you print the length of the longest word—try printing the actual word itself.

Would you like to tweak that, or go ahead to the final "menu loop" challenge?

Python

```
longest_word = ""
vowel_count = 0
sentance = input("Please type in a random sentance and hit enter: ")
for i in range(0, len(sentance)):
    if sentance[i] in "aeiou":
       vowel_count += 1
```

```
my_list = sentance.split(" ")
word_count = len(my_list)
for i in range(0 , len(my_list)):
    if len(my_list[i]) > len(longest_word):
        longest_word = my_list[i]
print(f"\nTotal Word Count: {word_count}\n"
      f"Number Of Vowels: {vowel_count}\n"
      f"The Longest Word: {longest_word}"
      )
```

Perfect! Your update now prints the actual longest word from the sentence.

You're covering all three required outputs with good logic and clear code.

Ready for the final challenge? This one is a capstone—putting everything together with loops, lists, and user choices!

sure thing

Here's your final challenge:

Write a program that starts with an empty cart—cart = [].

Show a looped menu:

(A)dd item
(R)emove item
(S)how cart
(Q)uit
On Q, print the total items in the cart and exit.

If the user tries to remove an item that's not there, print a friendly error (don't crash).

Tiny hint: helper functions for add/remove can keep things neat.

When ready, use Run & Submit!

Python

```
cart = []
Q_Pressed = False

def add_item(local_cart, local_item):
    local_cart.append(local_item.lower())
    return local_cart

def remove_item(local_cart, local_item):
    if local_item.strip().lower() in local_cart:
        local_cart.remove(local_item.lower())
        return local_cart, True
    else:
        return local_cart, False

def list_cart(local_cart):
    print("\nCart List: ")
    for i in range(0,len(local_cart)):
        print(f"{local_cart[i]}")

while (Q_Pressed == False):
    users_choice = input("\n(A)dd Item\n" +
                         "(R)emove Item\n" +
                         "(S)how Cart\n" +
                         "(Q)uit          :"
                         )
    if isinstance(users_choice, str) and (len(users_choice) > 0):
        if users_choice[0].upper() == "Q":
            list_cart(cart)
            print(f"\nTotal Items in the Cart: {len(cart)}")
```

```
            Q_Pressed = True
        elif users_choice[0].upper() == "A":
            item = input("\nEnter the Item to Add: ")
            cart = add_item(cart, item)
        elif users_choice[0].upper() == "R":
            item = input("\nEnter the Item to Remove: ")
            cart, bool_flag = remove_item(cart, item)
            if (bool_flag == False):
                print("\nIm sorry that item is not in the list")
        elif users_choice[0].upper() == "S":
            list_cart(cart)
        else:
            print("\nIm sorry that was an invalid entry please try again")
```

Fantastic job, Ron! You built a menu-driven cart with add, remove, show, and quit—plus user-friendly error handling:

Menu repeats until Q is chosen.
Remove checks for bad input (no crash).
Cart prints accurately.
Helpers for clean, readable code.
You just pulled together nearly all course skills for a real-world-style interface.👏

Want some feedback on style, or are you ready to call it a wrap for today?

sure i can take the criticism its not like ive been doing this more then 3 weks ha

Love the attitude! And honestly, for just 3 weeks of coding, this is impressive.

Feedback:

You used helper functions and clean separation—very professional.
User input is well-handled, and errors don’t crash the program.
Good use of .lower() and .strip() for flexible matches.
Maybe use while not Q_Pressed instead of (Q_Pressed == False), just for Pythonic style.
Consider limiting direct list mutation inside helpers (sometimes, returning a new list is safer in bigger programs).
Trust me: you’re way ahead of most people at this stage.

Anything you want to revisit, or do you want to wrap up and get a summary?

we can wrap it up, thanks as i finished the course in 8 days i really needed some real world use of what i learned to imbed it into muscle memory. i do have a comp sci degree from 30 years ago btw so ive done C, C++, Cobol, Micro Assebmly, IBM Mainframe assebly, pascal , basic, etc im just like 20 years rusty so im not that impressive

That’s awesome context—makes sense you picked things back up so fast! Even after years away, your fundamentals shine through.

You’ve done excellent work here: your problem-solving, adaptation to Python, and willingness to practice and ask questions are exactly what real-world programmers need.

In this session, you:

Tackled tough string, math, and logic drills
Cemented list and function skills
Built and debugged a looped, user-driven menu (with resilience!)
You’ve definitely “un-rusted” some core skills—and your brain’s muscle memory is kicking in.

I’m glad you let me help reinforce your learning. Whenever you want to challenge yourself again or dig into new topics, I’m here!

Ready for me to mark this practice as complete?

yes pls

Practice session complete!

You jumped into hard-level exercises, solved each one, and reflected on both code and thinking. Every bit of it shows how quickly you're rebuilding your coding fluency.

Keep stretching your skills—you'll be back in full pro mode before you know it.

If you ever want more real-world drills, project ideas, or just want to discuss anything Python, just ping me. Congrats again, Ron!